

Natural Language Interface for Queries on Databases with Sensitive Information

Suli Adeniye
Arizona State University
Tempe, USA

Faisal Al-Atawi
Arizona State University
Tempe, USA

Arunabha Sen
Arizona State University
Tempe, USA

Abstract—The analysis of large datasets poses significant challenges for users unfamiliar with structured query languages such as SQL or Cypher. These users, often from fields like social sciences or law enforcement, require intuitive and secure access to complex databases. Additionally, datasets in domains such as criminal justice, healthcare, and education often contain sensitive information, necessitating robust privacy protections. To address these challenges, we propose a privacy-preserving natural language interface for querying graph databases. Our framework integrates Optical Character Recognition (OCR) techniques to extract structured data from scanned and unstructured documents, storing it in a Neo4j graph database to efficiently model entity relationships. To facilitate user interaction, we leverage a Large Language Model (LLM)-powered query translation module that converts natural language queries into Cypher, eliminating the need for technical expertise.

To enhance privacy, our framework pseudonymizes sensitive entities using entity masking before query translation, ensuring that personally identifiable information within the user queries is never exposed to the LLM. Additionally, the Neo4j database is entirely separated from the LLM, meaning that the LLM only has access to the database schema and not any stored data. This ensures that query processing occurs in a controlled, authenticated environment, mitigating risks of data leakage. We also enforce schema-aware query translation, which significantly improves accuracy by guiding the LLM with structured database constraints.

We evaluate our system using real-world criminal records and the publicly available Text2Cypher dataset, demonstrating high logical and execution accuracy. Computational performance analysis shows that our privacy mechanisms introduce only minimal latency overhead (~5-8%), making them suitable for real-time applications. Our results confirm that this framework effectively balances accessibility, accuracy, and privacy, making it applicable to sensitive domains where secure, user-friendly data retrieval is essential.

Index Terms—sensitive records, graph database schema, LLM, user queries, query translation, information retrieval

I. INTRODUCTION

Traditional structured query languages (SQL, Cypher) require specialized knowledge, making them difficult for users without database expertise to retrieve information effectively. This limitation is particularly pronounced in domains such as law enforcement, where timely and accurate data retrieval is critical for decision making while ensuring data security and privacy [1], [2].

Graph databases, such as Neo4j, offer a more suitable alternative to relational databases when modeling complex relationships among entities such as offenders, crimes, and

locations. Their ability to efficiently represent and traverse relationships makes them ideal for law enforcement applications [3]. Graph-based approaches are increasingly being used to model complex relationships in interconnected datasets, providing efficient query mechanisms for applications. These methods enable structured representation of multi-entity relationships, supporting advanced network-based queries that uncover hidden patterns in criminal intelligence data [3]. However, querying graph databases requires knowledge of Cypher, a graph query language that remains inaccessible to non-technical users. Although previous work has advanced natural language to SQL (NL2SQL) [4], [5], fewer efforts have been directed toward enabling natural language to Cypher (NL2Cypher) while preserving data privacy [6], [7].

We introduce a framework that allows users to enter queries in natural language into a database interface and obtain desired responses in natural language, enabling intuitive access to structured data while ensuring data privacy. Our framework is realized by executing three primary tasks:

- 1) **Extracting Structured Data from Scanned Documents:** We received criminal records from one of the major US Metropolitan City Police Departments. Each record contains information about the individual who committed the crime and the list of crimes committed by the individual over a period of time. The format of the dataset is shown in Figure 1. The criminal records were provided to us as large scanned (image) PDF files. We utilize Optical Character Recognition (OCR) and Named Entity Recognition (NER) to extract structured data from scanned criminal history records. This process converts unstructured PDFs into tabular formats suitable for database creation. The table created from the PDF data extraction is shown in Table I.
- 2) **Constructing a Graph Database:** The resulting table is used to create a Neo4j graph database. Graph databases are selected over relational databases due to their superior performance in handling multi-hop queries, crime pattern analysis, and complex interconnections between offenders, crimes, and locations.
- 3) **Natural Language to Cypher Query Translation:** To facilitate intuitive interaction, we integrate a Large Language Model (LLM) that translates user queries into Cypher statements. To ensure privacy, the LLM



Fig. 1. Format of the dataset from Metropolitan City Police Department

TABLE I
EXTRACTED RECORDS FROM OFFENDERS' DOCUMENTS

Offender's Name	Gender	DoB	State	CrimeType	CrimeSeverity	CrimeCity	CrimeState	CrimeDate
O_1	M	...	CA	Pandering	3.1	LA	CA	Nov 3, 1992
O_2	F	...	NV	Driving under influence	2.2	LV	NV	...
O_2	F	...	NV	Domestic battery	2.3	LV	NV	...
...	...	...	...	...	...	...	...	...
O_n	M	...	...	Speed 10-20 mph above posted limit	1.5	...	...	...

operates under schema constraints and does not access raw sensitive data, mitigating risks of unintended data exposure.

We evaluate our system using the dataset received from the Metropolitan Police Department and also publicly available Text2Cypher dataset. Our results demonstrate high logical and execution accuracy.

The remainder of this paper is structured as follows: Section II reviews related work while the task execution steps of our framework are discussed in Section III. Section IV presents the system performance evaluation. Section V presents the applicability of our framework in other domains and Section VI concludes with future research directions.

II. RELATED WORK

This section reviews existing literature related to the three core areas of our framework: (1) extracting structured data from PDFs or scanned documents, (2) creating a graph database from structured data, and (3) implementing an LLM-powered translation module for querying graph databases. We discuss relevant work in each category and highlight how our approach advances the state-of-the-art.

A. Reading PDF Files to Generate Tables to Create the Database

Optical Character Recognition (OCR) plays a crucial role in converting scanned legal and criminal records into machine-readable formats. Traditional OCR tools such as Tesseract OCR [8] and Google Vision OCR have been widely used for text extraction, but often struggle with complex document structures, handwritten text, and noisy scanned data. ABBYY FineReader was chosen for this study due to its superior performance in handling complex legal documents and structured data extraction, as highlighted in previous evaluations of OCR technologies. Recent studies have explored

extracting structured data from law enforcement records and legal texts. Naik et al. [9] investigated Named Entity Recognition (NER) techniques for extracting legal entities from unstructured documents, demonstrating improved accuracy in entity classification. Rahman et al. [10] provided a comparative assessment of different information extraction techniques for document processing, emphasizing the need for tailored NLP pipelines for legal and law enforcement data. Smock et al. [11] developed PubTables-1M, a large-scale dataset for table extraction, advancing methodologies for extracting structured tabular data from complex document layouts.

Compared to prior works, our approach integrates OCR, Named Entity Recognition (NER) and structured data extraction techniques specifically tailored for criminal history records. This ensures improved accuracy and reliability when processing crime-related documents.

B. Creating a Database from the Table

Graph databases provide advantages over relational databases in modeling interconnected data, making them ideal for crime intelligence applications. Neo4j, leveraging labeled property graph (LPG) models, has gained popularity in this domain. Sharma and Sinha [12] emphasized the importance of schema-first approaches for structured data loading and integrity in LPG databases. Boudaoud et al. [13] developed a method for mapping relational databases to property graphs, ensuring information preservation and query consistency.

Putrama and Martinek [14] introduced an automated Neo4j graph construction framework, addressing metadata extraction and relationship preservation. Agrawal et al. [15] explored knowledge graph construction from unstructured texts, demonstrating the importance of entity-relation curation in crime data modeling. Our work extends these approaches by developing a domain-specific crime intelligence schema optimized for law enforcement, ensuring efficient multi-hop queries and data integrity.



Fig. 2. Preprocessing, conversion of extracted documents into graph database, and using cypher statements for information retrieval.

C. Translating Natural Language Queries into Database Query Languages

Translating natural language queries into structured database query languages (NL2SQL and NL2Cypher) has been an area of active research, with recent advancements leveraging large language models (LLMs), semantic parsing techniques, and schema-aware translation mechanisms. The primary challenge lies in interpreting ambiguous user queries, ensuring schema alignment, and handling query constraints, particularly in sensitive domains such as law enforcement, healthcare, and financial transactions.

Early efforts in natural language interfaces for databases primarily focused on SQL-based systems, where natural language inputs were mapped to structured SQL queries. Narechania et al. [5] developed DIY, an interactive debugging tool that allows users to assess and correct errors in NL2SQL-generated queries. Their work highlighted key challenges in query correctness assessment, error isolation, and user-driven refinements for ambiguous translations. Similarly, Fu et al. [16] introduced CatSQL, a system that improves NL2SQL performance by employing a reinforcement learning-based training strategy. The study demonstrated that domain-specific tuning enhances query generation accuracy and minimizes syntax errors in complex SQL queries.

Li et al. [17] analyzed the applicability of LLMs for database interfaces, revealing that while LLMs improve generalization across multiple datasets, schema-aware fine-tuning remains critical for precise query formulation. Their findings align with Fernandez et al. [1], who discussed the broader implications of LLMs in data management, emphasizing that despite the ability of LLMs to generate structured queries, they often require schema-grounding techniques to maintain execution accuracy.

While NL2SQL research has advanced significantly, NL2Cypher (natural language to Cypher translation for graph databases) remains underexplored. Graph-based querying presents additional challenges due to variable-length path traversals, entity disambiguation, and relationship constraints. Guo et al. [18] introduced a semantic parsing dataset for NL2Cypher, facilitating training models that convert natural language into Cypher queries. Their dataset aimed to bridge the gap in training resources for NL2Cypher systems and improve query generation accuracy.

Parikh et al. [19] developed Auto-Query, a simple NL2SQL generator for e-learning platforms, illustrating the impact of domain-specific constraints on translation quality.

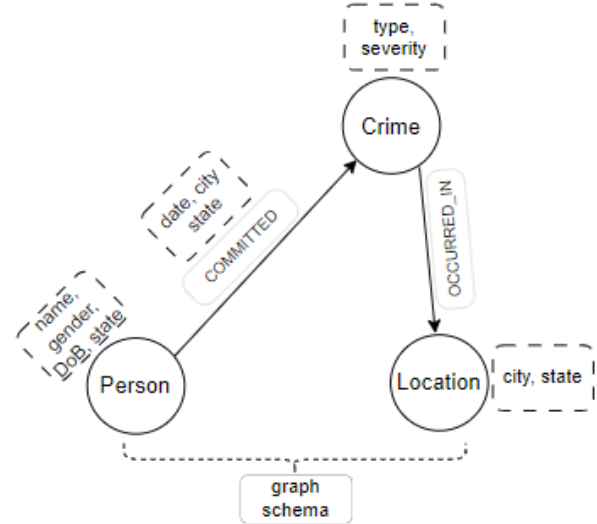


Fig. 3. The schema of the graph database.

Prakash et al. [20] examined the integration of LLMs into database education, demonstrating that explainability and schema-awareness mechanisms significantly improve the interpretability of generated queries. Uma et al. [21] proposed a formation pipeline for SQL from natural language using NLP, advocating for hybrid models combining rule-based parsing with LLM-driven translation.

Our framework advances these efforts by integrating:

- Schema-awareness to enhance Cypher query accuracy.
- Few-shot learning to refine query generation with minimal training data.
- Privacy safeguards to prevent the LLM from accessing raw sensitive data.

By incorporating domain-specific constraints and schema-linking mechanisms, our approach ensures robust and privacy-preserving query translation tailored to sensitive crime data.

III. TASK EXECUTION

This section details the execution of the key tasks involved in implementing the proposed framework: (1) Extracting structured data from raw criminal history documents, (2) Constructing a graph database from the structured data, and (3) Implementing an LLM-powered translation module to convert natural language queries into Cypher statements for querying the graph database. Additionally, we discuss maintaining data confidentiality, user query generation, query translation, resolution of ambiguous queries, and extending the framework to other domains.

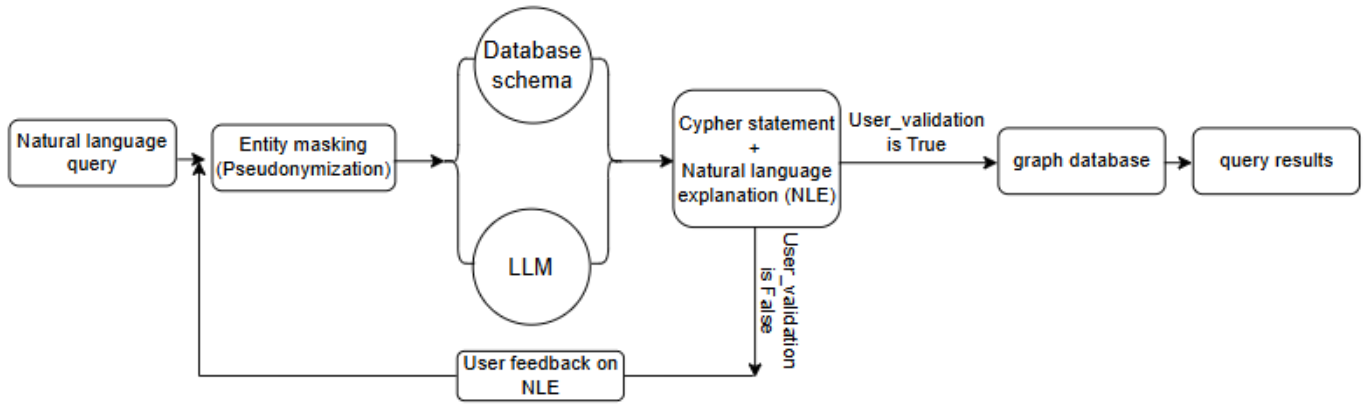


Fig. 4. The complete framework incorporating resolution of ambiguous queries and protection of sensitive data.

A. Execution of Task 1: Extracting Structured Data from Raw Criminal History Documents

The first step involves transforming unstructured criminal history records, typically in image-based PDF formats, into a structured tabular format suitable for database integration. The criminal history record documents, illustrated in Figure 1, contain the offender's biodata and their arrest records. Each arrest record includes the arrest date, law enforcement agency that conducted the arrest, one or more charges related to the offense, and the related court documents.

1) *Optical Character Recognition (OCR)*: To convert scanned documents into machine-readable text, we used ABBYY FineReader [22]. While ABBYY FineReader performed the initial text extraction, additional preprocessing was required to handle common OCR errors and misalignments.

2) *Text Correction and Preprocessing*: After OCR extraction, the text was reviewed for structural integrity. Common issues such as misalignments and character misrecognition were corrected using predefined rules. Any missing fields were either inferred based on context or flagged for manual review to ensure data completeness and accuracy.

3) *Parsing and Structuring*: A custom-built parser extracts essential information such as offender details, crime attributes, and crime locations. The extracted data is structured into a CSV file, as illustrated in Table I, providing a clean dataset for database construction.

B. Execution of Task 2: Constructing a Graph Database from the Structured Data

Once structured data was obtained, it was transformed into a graph database to enable efficient querying and analysis, as shown in Figure 2.

Graph Schema and Relationships

We formally define the graph database as a directed, labeled graph: $G = (V, E)$, where: where V represents the set of nodes (entities), including: V_{Person} (individual with attributes: name, DoB, gender, residence city, residence state). V_{Crime} (offense with attributes: crime type, severity, date). $V_{Location}$ (geographical crime location, with attributes: city, state). E represents the set of relationship between these entities, specifically, $E_{COMMITTED}$ is a directed edge from V_{Person} to

V_{Crime} with attributes crime type, crime severity, and crime date. $E_{OCCURRED_IN}$ is a directed edge from V_{Crime} to $V_{Location}$.

The database schema consisted of three primary entities: Person (representing offenders), Crime (containing offense details), and Location (where the crime occurred). These entities were linked through relationships, with **COMMITTED** connecting a Person to a Crime and **OCCURRED_IN** linking Crime to Location.

Graph Database Construction A Neo4j database was populated using a custom-built script that iterates through the CSV records, ensuring correct entity-relationship mappings. Figure 3 presents the schema used for the database. This structured approach optimized performance for complex queries involving multiple relationships, enabling more effective analysis of criminal patterns and associations.

C. Execution of Task 3: Implementing an LLM-Powered Translation Module

The final task involved integrating a large language model (LLM) to translate natural language queries into Cypher, enabling non-technical users to interact with the graph database. Figure 4 illustrates the complete framework.

- **Translation Context:** To generate accurate Cypher queries, the LLM was provided with a structured graph schema and example user queries and corresponding Cypher statements. These guided the model in understanding schema-specific patterns.

- **Maintaining Data Confidentiality:**

- (i) Restricted LLM Access** The LLM operates solely on the schema and predefined example user queries/Cypher statements, ensuring it has no access to real database records.
- (ii) Local Database Separation:** The graph database remains local and separate from the LLM, with authentication enforcing secure access control.
- (iii) Entity Masking:** Personally identifiable entities (e.g., names, locations, dates) are replaced with placeholders before queries are sent to the LLM. Original entities are reinserted before execution, ensuring sensitive data is protected throughout the process. For example, a user query such as: "Was Bob Mallory involved in 'sex trafficking'"

offense that occurred in Phoenix from January 2020 through July 2020?" is transformed to "Was [PERSON_1] involved in 'sex trafficking' offense that occurred in [CITY_1] from [DATE_1] through [DATE_2]?" before being processed by the LLM. This ensures that the LLM never sees or learns any sensitive information and prevents it from correlating entities across multiple queries.

- **Query Disambiguation:** Ambiguous queries are refined interactively, allowing users to iteratively validate or modify the query based on Natural Language Explanations (NLEs) on the translated Cypher query before execution.
- **Execution and Query Retrieval:** Once validated, translated Cypher statements are executed on the Neo4j database to retrieve structured crime-related data. This ensures an intuitive and secure interaction without requiring users to learn Cypher syntax. This approach balances accessibility, security, and accuracy, ensuring privacy while enabling efficient querying of sensitive datasets.

IV. SYSTEM PERFORMANCE EVALUATION

This section evaluates the effectiveness of our framework in translating natural language queries into Cypher statements for retrieving relevant information from a graph database. Our evaluation focuses on logical accuracy, execution accuracy, translation and execution times, and the framework's generalizability across different datasets.

A. Evaluation Methodology

We used two datasets to assess our system:

- **Sensitive Crime Records Dataset** – A real-world dataset modeled as a graph database, containing offender information, crimes, and associated locations.
- **Text2Cypher Dataset** – A publicly available dataset used to benchmark natural language-to-Cypher translations across multiple domains.

Each natural language query was paired with its expected Cypher statement (ground truth Cypher) and the expected database results (ground truth results). The system-generated Cypher queries were evaluated for logical accuracy by comparing them to the ground truth Cypher and for execution accuracy by comparing the retrieved results with the ground truth results.

B. User Queries Generation

Real-world queries from law enforcement were categorized into:

- **Atomic Queries:** Simple entity-based queries. Examples are "What is the gender distribution of offenders in the dataset?", "What is the count of offenders per state of residence?", "What are the most common types of crimes?".
- **Complex Queries:** Queries involving multiple entities, relationships, and conditions. For instance, "Which offenders committed crimes outside their state of residence?" "Find offenders who committed crimes in multiple states and list their crime details.", "Find offenders

who committed crimes involving 'sex trafficking of child under 16' and their severities"

- **Ambiguous Queries:** Queries which require disambiguation or additional refinement. For example, "What are the patterns of reoffending for offenders in Nevada?". Ambiguity: Does "patterns of reoffending" refer to the frequency of reoffending, or the type of crimes committed upon reoffending, or the locations of reoffenses? "Who are the most dangerous offenders?". Ambiguity: What criteria determine "dangerous"? Severity level of crimes, or frequency of offenses, or specific types of crimes like domestic violence or trafficking?

The evaluation covered 120 atomic queries, 200 complex queries, and 100 ambiguous queries, collected in collaboration with law enforcement officers. Due to space constraints, only 12 complex queries are presented in Table II, while the full list is available on our GitHub repository. All simple queries were tested separately (not included in the paper due to space limitations), achieving 100% logical and execution accuracy. The lower count of ambiguous queries reflects practical challenges in obtaining sufficient ambiguous query samples from law enforcement professionals. Future work will focus on expanding this category.

C. Evaluation Metrics

To quantitatively assess our framework, we employed the following metrics, adapted from prior work on natural language-to-database query translation [16], [6].

- **Logical Accuracy:** Measures whether the generated Cypher query is semantically equivalent to the ground truth Cypher. A query is considered correct if it retrieves the same results as the reference Cypher, even if syntactic variations exist. Formally, for a set of queries:

$$LA_{query} =$$

$$\begin{cases} 1 & \text{if generated Cypher} \equiv \text{ground truth Cypher,} \\ 0 & \text{otherwise.} \end{cases}$$

The system-wide logical accuracy is given by:

$$LA_{system} = \frac{\sum_i^N LA_i}{N}, \text{ where } N \text{ is the total number of queries across each of the simple, or complex, or ambiguous query categories.}$$

- **Execution Accuracy:** Measures whether the results retrieved by the generated Cypher query exactly match those from the ground truth Cypher execution in the database:

$$EX_{query} =$$

$$\begin{cases} 1 & \text{if retrieved results from the generated Cypher} \\ & \text{match those from the ground truth Cypher} \\ 0 & \text{otherwise.} \end{cases}$$

The system-wide accuracy is given by:

$$EX_{system} = \frac{\sum_i^N EX_i}{N}, \text{ where } N \text{ is the total number of queries across each of the simple, or complex, or ambiguous query categories.}$$

TABLE II
COMPLEX USER QUERIES

Queries about offenders and committed crimes	Queries about offenders and crime locations	Queries about committed crimes and crime locations	Queries about offenders, committed crimes, and crime locations
List names, gender, age, and states of offenders that committed crimes containing "kidnaping" in their crime type.	List offenders who committed crimes in two or more cities within the same state.	What locations have the highest average crime severity?	What is dominant state of out-of-state traffickers arrested for crimes involving sex trafficking in Las Vegas from January 2020 through July 2020?
What is the distribution of crime severity levels among male offenders?	Which offenders have crimes recorded in multiple years and locations?	Find the cities where sex trafficking of children under 18 is most common	Which offenders committed crimes involving 'assault' in multiple cities?
Do they have criminal charges in multiple states? If yes, where?	Find the states where crimes were committed by offenders who do not reside there.	Which crimes were recorded in Boston, Massachusetts, and in Phoenix, Arizona?	Who are the offenders with crimes in multiple cities in Oregon?

TABLE III
EVALUATION OF RESULTS (CRIME RECORDS DATASET)

Evaluation Metric	Queries about offenders and committed crimes	Queries about offenders and crime locations	Queries about committed crimes and crime locations	Queries about offenders, committed crimes, and crime locations
Logical Accuracy, without graph schema (%)	54	90	90	80
Logical Accuracy, with graph schema (%)	92	100	100	94
Execution Accuracy (%)	92	100	100	94
Logical Accuracy with Entity Masking (%)	92	96	98	92

TABLE IV
EVALUATION OF RESULTS (TEXT2CYPHER DATASET)

Evaluation Metric	Crime and offenses related queries	Business reviews related queries	Computer network connectivity related queries	Product procurement management related queries
Logical Accuracy - without graph schema (%)	77	79	53	62
Logical Accuracy - with graph schema (%)	98	97	94	98

TABLE V
QUERY TRANSLATION AND EXECUTION TIMES (IN SECONDS)

Evaluation Metric	Queries about offenders and committed crimes	Queries about offenders and crime locations	Queries about committed crimes and crime locations	Queries about offenders, committed crimes, and crime locations
Query translation (with no privacy measure)	1.61	1.69	1.68	1.60
Query translation (with entity masking)	1.65	1.78	1.69	1.78
Query execution	2.25	2.15	2.11	2.17

D. Results And Analysis

1) *Performance on the Sensitive Crime Records Dataset:*
Table III presents the evaluation results for queries executed on the Sensitive Crime Records Dataset. Our framework demonstrated strong logical accuracy, particularly when the graph schema was provided to the LLM as context. Without schema constraints, logical accuracy for complex queries varied significantly across different categories: 54% for offender-related queries, 90% for offender-location queries, 90% for crime-location queries, and 80% for offender-crime-location

queries. However, when schema constraints were explicitly included, logical accuracy improved substantially to 92%, 100%, 100%, and 94%, respectively. A similar pattern was observed for execution accuracy, where schema awareness resulted in significantly better accuracy rates of 92%, 100%, 100%, and 94% across these categories. Furthermore, we tested the effect of entity masking, which replaces identifiable entities in queries before translation to Cypher. Results showed that while entity masking slightly reduced accuracy (e.g., 98% for crime-location queries instead of 100%), it remained

highly effective in protecting sensitive information without severely impacting performance. These results reinforce the importance of schema-awareness and privacy-preserving techniques, ensuring that the system maintains high accuracy while preventing potential data exposure.

2) *Performance on the Text2Cypher Dataset:* We further evaluated our framework on the Text2Cypher dataset, a publicly available benchmark dataset covering multiple domains. Results are summarized in Table IV. With schema guidance, the logical accuracies for Crime-related, Business reviews, Network Connectivity-related, and Product Procurement-related queries are: 98%, 97%, 94%, and 98% respectively. Without schema guidance, performance dropped across all categories, with network connectivity queries being particularly affected (53%). This suggests that the framework generalizes well to non-criminal domains when schema constraints are available. The drop in accuracy in schema-less conditions highlights the importance of schema grounding in LLM-based query translation.

3) *Computational Performance: Query Translation and Execution Time:* To evaluate the computational efficiency of our framework, we measured both query translation time (LLM processing) and query execution time (retrieval from the database). To ensure reliability, we ran each query three times, recorded the maximum time, and then computed the average across 50 queries in each category of Table II. The results are summarized in Table V.

- **Query Translation Time:** without privacy measure (direct NL2Cypher translation), translation time ranged between 1.60s - 1.69s, depending on query complexity. However, translation time increased slightly, ranging from 1.65s - 1.78s due to the additional processing required for entity recognition and masking. Furthermore, more complex queries (e.g., involving multiple relationships) took slightly longer to translate compared to simpler queries.
 - **Query Execution Time:** Execution times were consistent across all query types, averaging $\sim 2.1s - 2.25s$. Entity masking did not introduce any significant overhead during execution since the masked entities were replaced with their original values before querying the database.
 - **Observations and Trade-offs:** Entity masking introduces a minor latency overhead ($\sim 5-8\%$), but significantly enhances privacy protection. Query execution is not a bottleneck, as the primary computational cost lies in LLM-based query translation rather than Cypher query execution. Using the maximum time from three runs for each query ensures a robust latency estimate, accounting for possible fluctuations in processing time.
- These results demonstrate that our framework achieves a balance between efficiency, accuracy, and privacy, ensuring minimal computational overhead while maintaining high query translation fidelity.

4) *Experiment Environment:* All experiments were conducted on a local computing environment with the following specifications: Intel i7 1.80GHz CPU, 16GB RAM, Windows

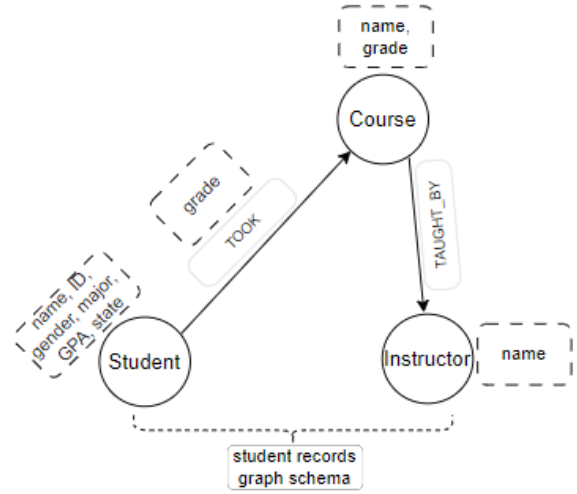


Fig. 5. Proposed schema for the student records graph database.

10 Enterprise, and GPT-4o mini (LLM). The query translation component leveraged the GPT-4o mini's API for processing, while all other computations, including entity masking, Cypher query generation, and execution, were performed locally. The Neo4j graph database was also hosted locally, ensuring secure and efficient query execution without network latency.

5) *Comparative Analysis Across Datasets:* Analyzing the results from both datasets, we observe key trends:

- Schema awareness is crucial: The framework performs best in domains where structured schema information is available.
- Logical accuracy significantly improves with schema guidance, reinforcing the importance of schema-aware few-shot learning.
- Privacy mechanisms (entity masking) protect sensitive user queries while maintaining high accuracy, with only minor computational overhead. Furthermore, the system system generalizes well beyond criminal records datasets.

These findings highlight the robustness of our framework in translating natural language to Cypher, while demonstrating the trade-offs between accuracy, efficiency, and privacy.

V. APPLICABILITY OF THE FRAMEWORK IN OTHER DOMAINS

Although the framework presented in this paper was motivated by query processing problems in criminal records data, the framework can be adapted for other domains such as for patient records data in healthcare and student record data in an educational institution. We elaborate on this idea in educational institution setting using sample student records shown in Table VI, corresponding graph schema illustrated in Figure 5 and some sample queries presented in Table VII. Similar illustration can be made for patient health records.

VI. CONCLUSION

We present a privacy-preserving natural language interface for querying graph databases in sensitive domains. Our framework integrates LLM-powered query translation with schema

TABLE VI
STUDENT ACADEMIC RECORDS, XYZ COLLEGE

Student's Name	Student ID	Gender	Major	GPA	Year of Graduation	Course	Grade	Instructor
S_1	121116	F	Computer science	3.8	Fall 2022	Algorithms	A	Dr. A. Joe
S_2	133452	...	Chemical engr'g	3.5	Spring 2023	Thermodynamics I	A+	Prof. C. Lee
...	...	...	...	...	...	...	...	...
S_n	121120	M	Civil engr'g	3.2	Spring 2024	Strength of materials	B+	Dr. B. Wilkis

TABLE VII
SAMPLE USER QUERIES FOR STUDENT ACADEMIC RECORDS

#	User Queries
1	List the students who scored the highest grade in courses taught by Prof. Lee
2	Show the grade distributions for courses taught by Dr. Smith in 2022
3	Identify the courses where students scored the most A+ grades.
4	Find the courses that have been taught by more than one instructor

guidance and entity masking, ensuring high accuracy while protecting personally identifiable information. Evaluations on real-world crime records and Text2Cypher confirm its effectiveness, with minimal computational overhead ($\sim 5\text{-}8\%$).

To enhance usability, we introduce interactive query refinement for ambiguous queries, improving clarity before translation. Our findings highlight the importance of schema awareness and predefined database constraints in boosting accuracy. Future work will focus on adversarial robustness and adaptive query ranking. This research advances secure, efficient, and accessible natural language querying, bridging the gap between non-technical users and complex graph databases.

VII. ACKNOWLEDGMENTS

The authors are grateful for the contributions of Kartikeya Kansal, Christopher Sumnicht, Dev Dipak Patel, and Varshita Mogalapalli, all of Networks Existence (NetXT) Lab, Arizona State University.

REFERENCES

- [1] R. C. Fernandez, A. J. Elmore, M. J. Franklin, S. Krishnan, and C. Tan, "How large language models will disrupt data management," *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3302–3309, 2023.
- [2] A. Quamar, V. Efthymiou, C. Lei, F. Özcan, *et al.*, "Natural language interfaces to data," *Foundations and Trends® in Databases*, vol. 11, no. 4, pp. 319–414, 2022.
- [3] S. R. Muramudalige, B. W. Hung, A. P. Jayasumana, and I. Ray, "Investigative graph search using graph databases," in *2019 first international conference on graph computing (gc)*, pp. 60–67, IEEE, 2019.
- [4] R. Kaoshik, R. Patil, S. Agarawal, N. Jain, and M. Singh, "Acl-sql: Generating sql queries from natural language," in *Proceedings of the 3rd ACM India Joint International Conference on Data Science & Management of Data (8th ACM IKDD CODS & 26th COMAD)*, pp. 423–423, 2021.
- [5] A. Narechania, A. Fourney, B. Lee, and G. Ramos, "Diy: Assessing the correctness of natural language to sql systems," in *Proceedings of the 26th International Conference on Intelligent User Interfaces*, pp. 597–607, 2021.
- [6] A. Guo, X. Li, G. Xiao, Z. Tan, and X. Zhao, "Spcql: A semantic parsing dataset for converting natural language into cypher," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pp. 3973–3977, 2022.
- [7] N. de Jong, "Neodash - neo4j dashboard builder." <https://github.com/nielsdejong/neodash>, 2022. Accessed: 2025-03-15.
- [8] R. Smith, "An overview of the tesseract ocr engine," in *Ninth international conference on document analysis and recognition (ICDAR 2007)*, vol. 2, pp. 629–633, IEEE, 2007.
- [9] V. Naik, P. Patel, and R. Kannan, "Legal entity extraction: An experimental study of ner approach for legal documents," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 3, 2023.
- [10] A.-u. Rahman, D. Musleh, M. Nabil, H. Alubaidan, M. Gollapalli, G. Krishnasamy, D. Almoqbil, M. A. A. Khan, M. Farooqui, M. I. B. Ahmed, *et al.*, "Assessment of information extraction techniques, models and systems," *Mathematical Modelling of Engineering Problems*, vol. 9, no. 3, 2022.
- [11] B. Smock, R. Pesala, and R. Abraham, "Pubtables-1m: Towards comprehensive table extraction from unstructured documents," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4634–4642, 2022.
- [12] C. Sharma and R. Sinha, "A schema-first formalism for labeled property graph databases: Enabling structured data loading and analytics," in *Proceedings of the 6th ieee/acm international conference on big data computing, applications and technologies*, pp. 71–80, 2019.
- [13] A. Boudaoud, H. Mahfoud, and A. Chikh, "Towards a complete direct mapping from relational databases to property graphs," in *International Conference on Model and Data Engineering*, pp. 222–235, Springer, 2022.
- [14] I. M. Putrama and P. Martinek, "An automated graph construction approach from relational databases to neo4j," in *2022 IEEE 22nd International Symposium on Computational Intelligence and Informatics and 8th IEEE International Conference on Recent Achievements in Mechatronics, Automation, Computer Science and Robotics (CINTI-MACRo)*, pp. 000131–000136, IEEE, 2022.
- [15] G. Agrawal, Y. Deng, J. Park, H. Liu, and Y.-C. Chen, "Building knowledge graphs from unstructured texts: Applications and impact analyses in cybersecurity education," *Information*, vol. 13, no. 11, p. 526, 2022.
- [16] H. Fu, C. Liu, B. Wu, F. Li, J. Tan, and J. Sun, "Catsql: Towards real world natural language to sql applications," *Proceedings of the VLDB Endowment*, vol. 16, no. 6, pp. 1534–1547, 2023.
- [17] G. Li, X. Zhou, and X. Zhao, "Llm for data management," *Proceedings of the VLDB Endowment*, vol. 17, no. 12, pp. 4213–4216, 2024.
- [18] Y. Guo, J. Liu, W. Tang, and C. Huang, "Exsense: Extract sensitive information from unstructured data," *Computers & Security*, vol. 102, p. 102156, 2021.
- [19] P. Parikh, O. Chatterjee, M. Jain, A. Harsh, G. Shahani, R. Biswas, and K. Arya, "Auto-query-a simple natural language to sql query generator for an e-learning platform," in *2022 IEEE Global Engineering Education Conference (EDUCON)*, pp. 936–940, IEEE, 2022.
- [20] K. Prakash, S. Rao, R. Hamza, J. Lukich, V. Chaudhari, and A. Nandi, "Integrating llms into database systems education," in *Proceedings of the 3rd International Workshop on Data Systems Education: Bridging education practice with education research*, pp. 33–39, 2024.
- [21] M. Uma, V. Sneha, G. Sneha, J. Bhuvana, and B. Bharathi, "Formation of sql from natural language query using nlp," in *2019 International Conference on Computational Intelligence in Data Science (ICCIDS)*, pp. 1–5, IEEE, 2019.
- [22] M. Cuper, C. van Dongen, and T. Koster, "Unraveling confidence: examining confidence scores as proxy for ocr quality," in *International Conference on Document Analysis and Recognition*, pp. 104–120, Springer, 2023.